MediVerse AI: A Multi-Agent Healthcare Intelligence Platform for Unified Patient Health Records and Clinical Decision Support

23AID498 Project Phase 1(Review-2)

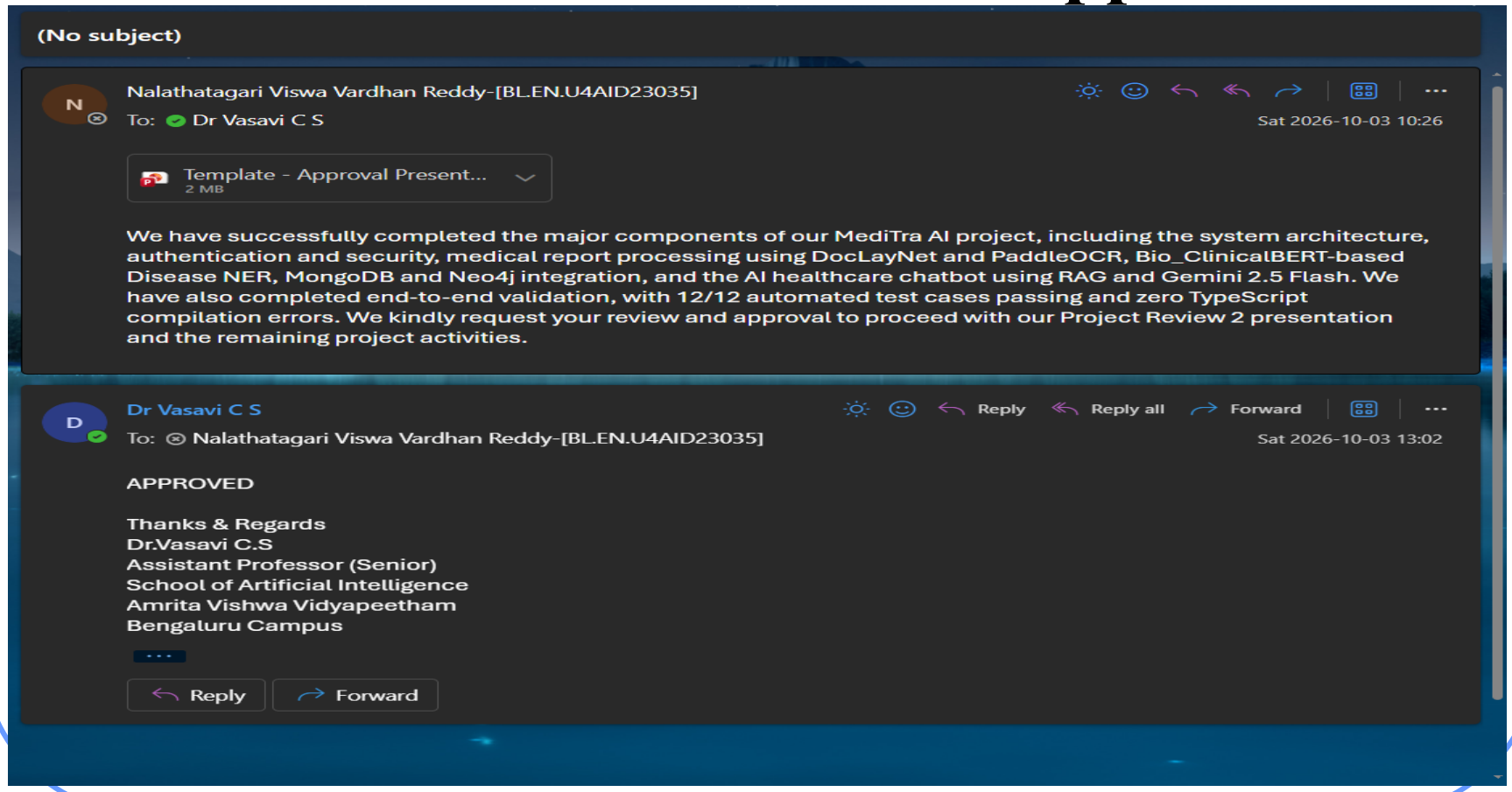
Team ID: 23UG088

Panel No.: 7

S.No	Reg.No	Name of the Student	Branch
1	BL.EN.U4AID23012	C Venkata Siva Sai Raju	AID
2	BL.EN.U4AID23022	K Subhash Chandra	AID
3	BL.EN.U4AID23035	N Viswa Vardhan	AID
4	BL.EN.U4AID23040	P Mithun Karthikeya	AID

Project Guide: Dr.Vasavi C.S

Screenshot of the Guide's Approval



(No subject)

Nalathatagari Viswa Vardhan Reddy-[BL.EN.U4AID23035]
To: Dr Vasavi C S
Sat 2026-10-03 10:26

Template - Approval Present...
2 MB

We have successfully completed the major components of our MediTra AI project, including the system architecture, authentication and security, medical report processing using DocLayNet and PaddleOCR, Bio_ClinicalBERT-based Disease NER, MongoDB and Neo4j integration, and the AI healthcare chatbot using RAG and Gemini 2.5 Flash. We have also completed end-to-end validation, with 12/12 automated test cases passing and zero TypeScript compilation errors. We kindly request your review and approval to proceed with our Project Review 2 presentation and the remaining project activities.

Dr Vasavi C S
Reply Reply all Forward
To: Nalathatagari Viswa Vardhan Reddy-[BL.EN.U4AID23035]
Sat 2026-10-03 13:02

APPROVED

Thanks & Regards
Dr.Vasavi C.S
Assistant Professor (Senior)
School of Artificial Intelligence
Amrita Vishwa Vidyapeetham
Bengaluru Campus

Reply Forward

Problem Definition

•Medical records are scattered across different reports and hospitals.

•Important information such as diseases, medicines, allergies, and lab results is difficult to organize.

•Manual analysis of medical reports is time-consuming.

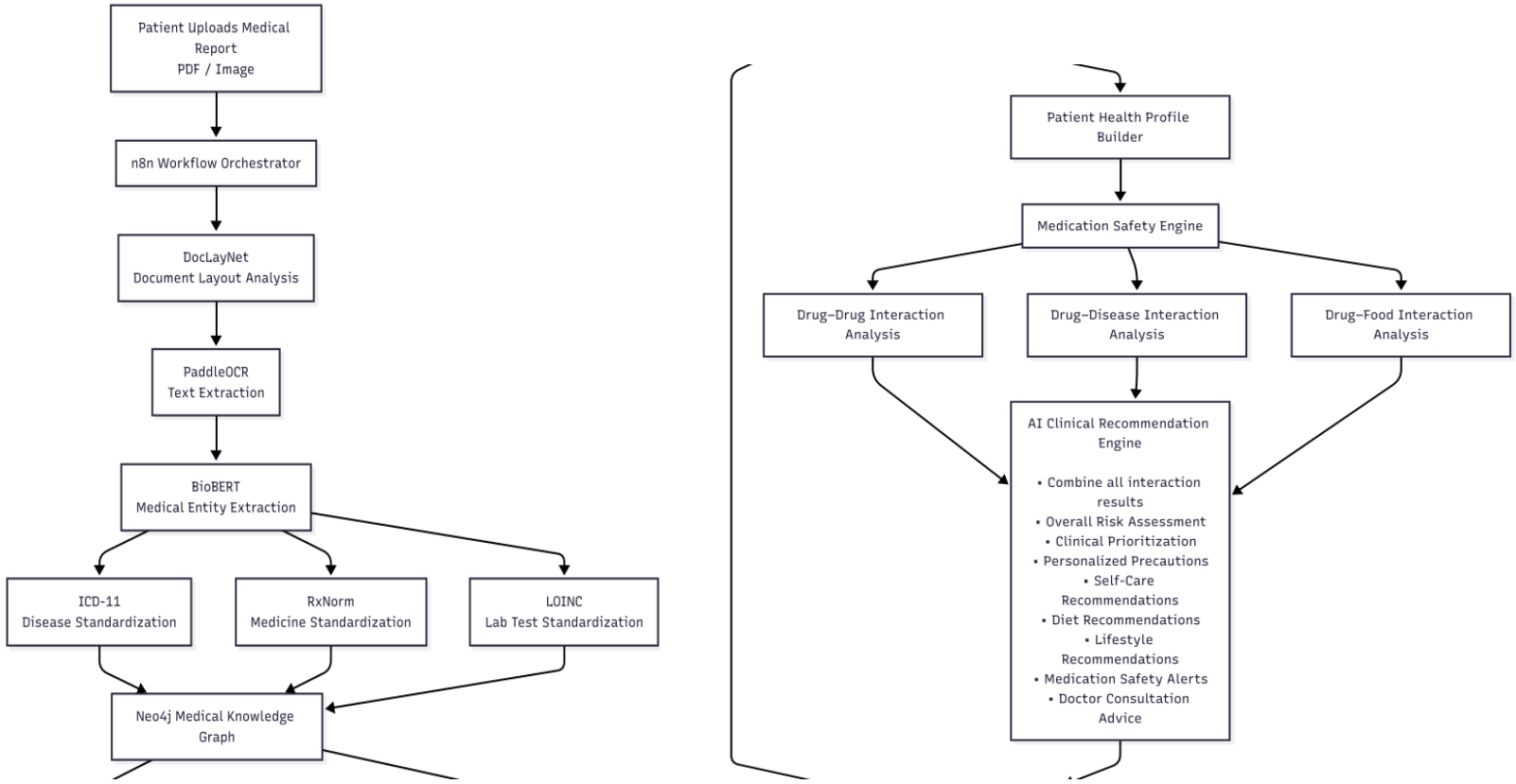
• Medication risks such as drug–drug, drug–disease, and drug–food interactions may be difficult to identify together.

- Existing AI assistants often lack complete patient-specific medical context.

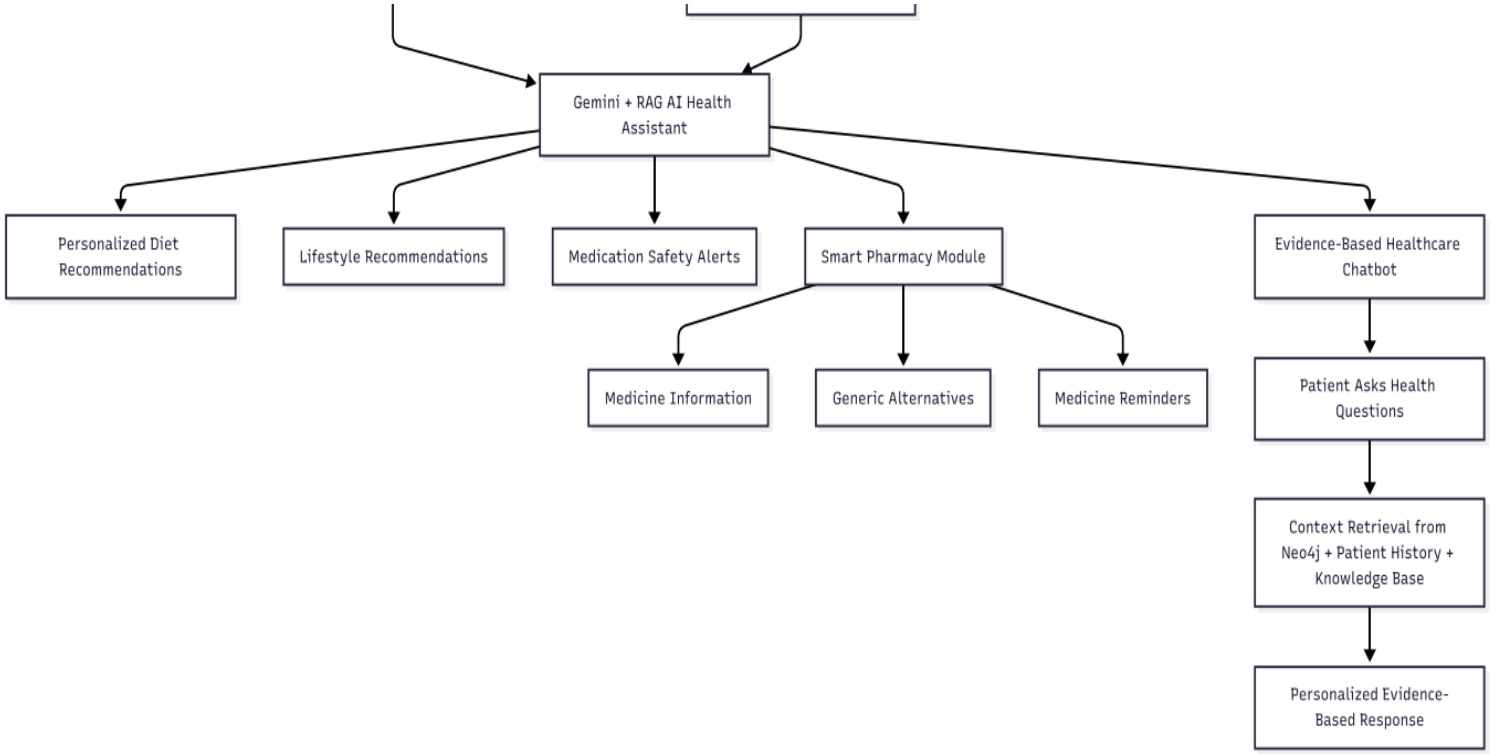
Project Objectives

- Automatically extract medical information from uploaded reports.
- Create a unified patient health profile.
- Build a Medical Knowledge Graph connecting patient health information.
- Identify drug–drug, drug–disease, and drug–food interactions.
- Provide personalized healthcare assistance using AI and RAG.
- Provide medicine information, reminders, and pharmacy assistance.

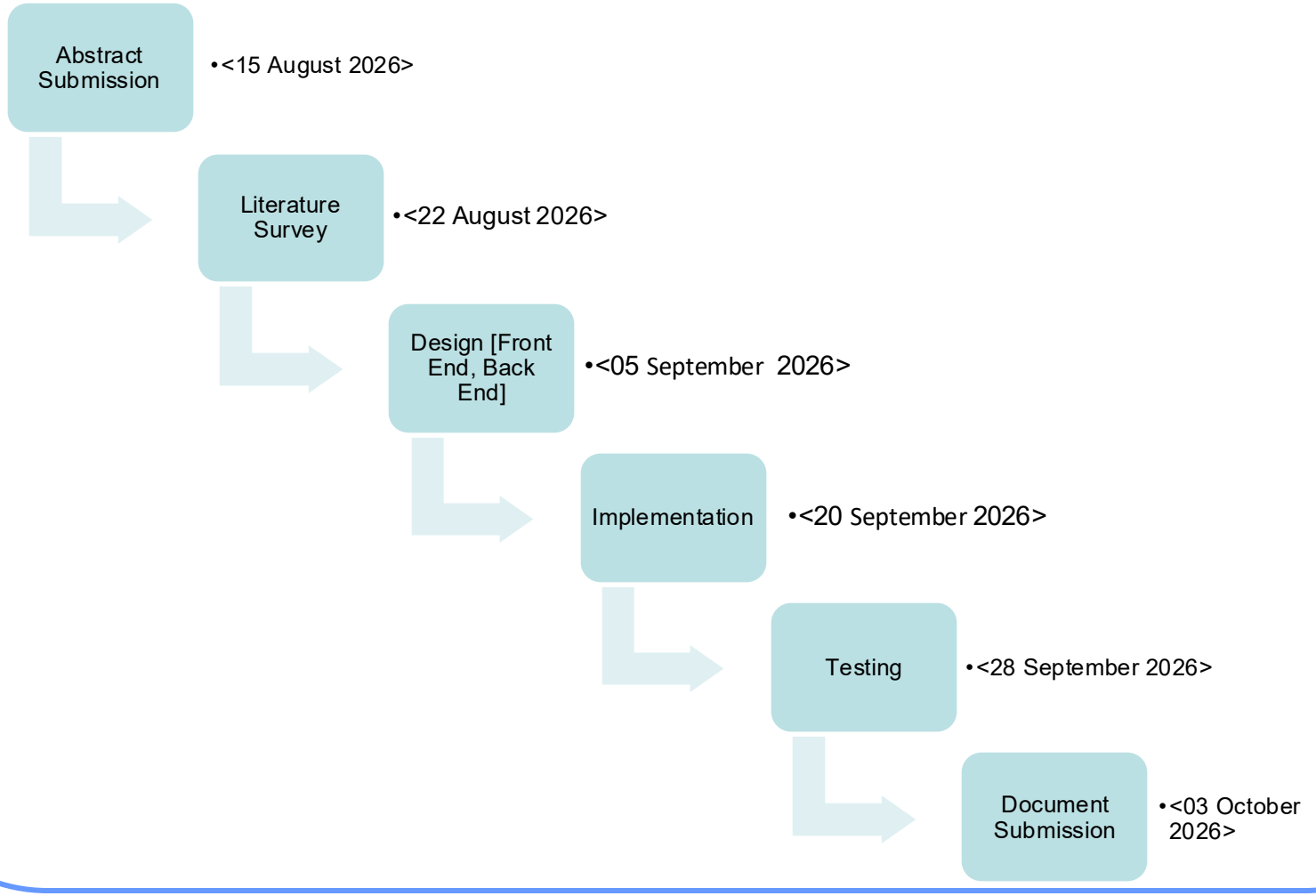
Architecture of the Proposed System



Architecture of the Proposed System



Timeline


Abstract Submission
•<15 August 2026>
Literature Survey
•<22 August 2026>
Design [Front End, Back End]
•<05 September 2026>
Implementation
•<20 September 2026>
Testing
•<28 September 2026>
Document Submission
•<03 October 2026>

Implemented System Architecture & Data Flow

•React + TypeScript frontend handles user authentication, medical report upload, and healthcare chatbot interaction.

•FastAPI backend provides REST APIs and connects the frontend with AI/ML services and workflows.

•n8n workflows orchestrate medical report processing using DocLayNet and PaddleOCR.

•Bio_ClinicalBERT extracts disease entities from processed medical reports and stores them in MongoDB.

•RAG + Gemini 2.5 Flash uses patient context from MongoDB and Neo4j to provide context-aware healthcare responses with a safety layer.

Phase 1 – Technical Progress

•Developed the React + TypeScript frontend and FastAPI backend for the healthcare platform.

•Implemented secure authentication and role-based access for patients, doctors, and admins.

•Completed Workflow 1 using DocLayNet + PaddleOCR for medical report processing and text extraction.

•Implemented Workflow 2 using fine-tuned Bio_ClinicalBERT for disease entity extraction.

•Integrated MongoDB, Neo4j, n8n, and AI services to support patient data processing and healthcare assistance.

Authentication & Security Implementation

•Integrated Firebase Admin SDK for secure user identity verification.

•Implemented JWT session-based authentication with secure HttpOnly cookies.

•Added role-based access control for Patients, Doctors, and Admins.

•Protected backend APIs to allow access only to authenticated and authorized users.

•Securely forwards authentication tokens between FastAPI and n8n without storing them in MongoDB.

Workflow 1 – Medical Report Processing Pipeline

- Accepts uploaded medical reports with file-size, file-type, and security validation.
- Converts uploaded PDF pages into PNG images for further processing.
- Uses DocLayNet to identify and separate important sections of the medical report.
- Applies PaddleOCR to extract and clean text from the detected sections.
- Stores the cleaned and structured medical report data in MongoDB, with the workflow orchestrated through n8n

Workflow 2 – Bio_ClinicalBERT Disease NER

•Uses Bio_ClinicalBERT, fine-tuned on the NCBI Disease Corpus, for disease extraction.

•Takes the cleaned OCR text from Workflow 1 as input for medical entity processing.

•Extracts disease names, confidence scores, and character positions from medical reports.

•Uses sliding-window processing to handle long medical reports without losing context.

•Stores extracted disease entities in the MongoDB medical_entities collection for further use.

Database & Neo4j Integration

•Integrated MongoDB Atlas to store users, reports, medical entities, and chatbot conversations.

•Stores processed OCR data in the medical_reports collection.

•Stores Bio_ClinicalBERT extracted diseases in the medical_entities collection.

•Integrated Neo4j Graph Database to retrieve patient disease and medication relationships.

•Implemented fallback support so the system can continue functioning if Neo4j is temporarily unavailable.

AI Healthcare Chatbot & Safety Layer

- Uses Gemini 2.5 Flash with RAG to generate context-aware healthcare responses.
- Retrieves patient context from MongoDB and Neo4j before generating responses.
- Maintains multi-turn conversation history for continuous patient interaction.
- Includes emergency red-flag detection and medication-safety restrictions.
- Provides a clinical disclaimer to clearly indicate that responses are supportive and not a medical diagnosis.

System Implementation & Demonstration

- Developed a working React + TypeScript web interface connected with the FastAPI backend.
- Demonstrated medical report upload, processing, OCR extraction, and structured storage.
- Successfully extracted disease entities using the fine-tuned Bio_ClinicalBERT model.
- Integrated MongoDB and Neo4j for patient data storage and contextual information retrieval.
- Demonstrated the AI healthcare chatbot using patient context, RAG, Gemini, and healthcare safety checks.

Results

```
TASK:
Disease Named Entity Recognition
...
DATASET:
NCBI Disease Corpus

TRAIN DOCUMENTS:
593

VALIDATION DOCUMENTS:
100

TEST DOCUMENTS:
100

LABELS:
0 = O
1 = B-DISEASE
2 = I-DISEASE

TRAINING:
completed

BEST VALIDATION F1:
0.7417027417027416

TEST PRECISION:
0.7016129032258065

TEST RECALL:
0.8656716417910447

TEST F1:
0.7750556792873052

TEST ACCURACY:
0.9672808132147395

MODEL DIRECTORY:
/content/medical_ner_final/

GPU:
Tesla T4

ZIP:
/content/medical_ner_final.zip
================================================================================
Scope: disease mention extraction only. This model does not diagnose or prescribe.
```

Fig1:- Bio_ClinicalBERT Disease NER – Model Performance Results

Results

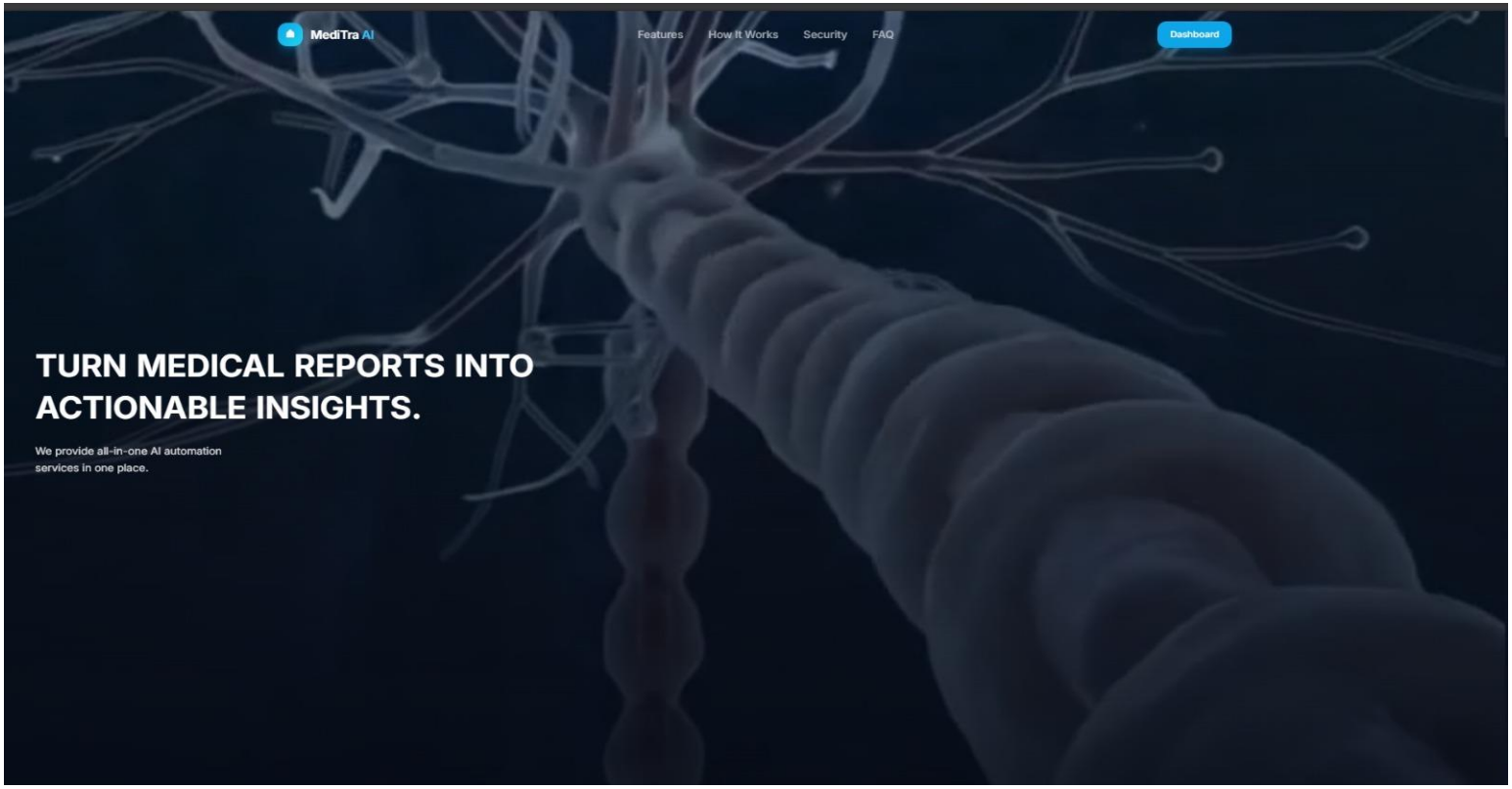


Fig2:- MediTra AI – Implemented Web Application Interface

Results

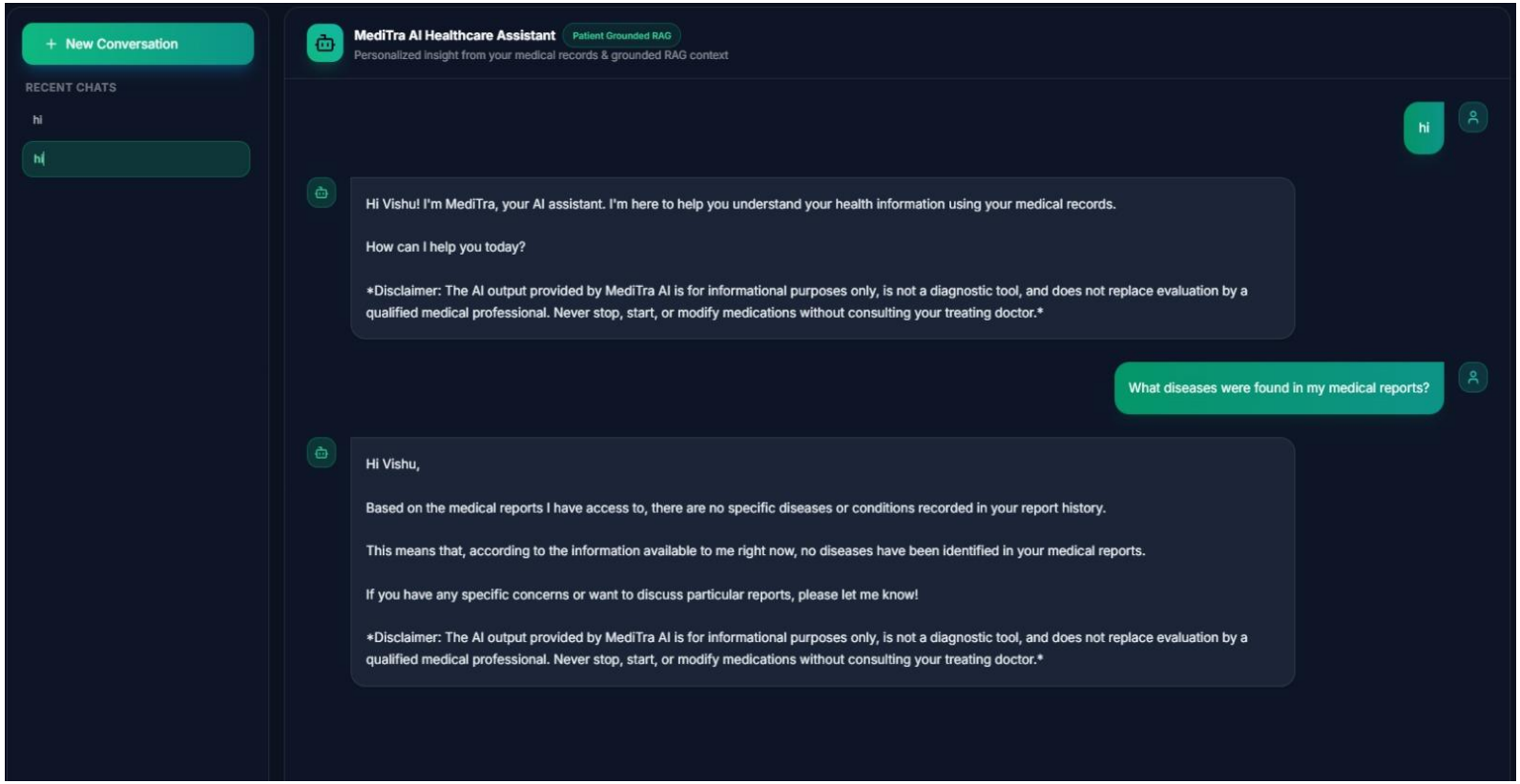


Fig3:-Patient-Grounded RAG Healthcare Chatbot – Response Result

Objective Achievement & Current Project Status

•Completed the full-stack foundation using React, TypeScript, FastAPI, and n8n.

•Successfully implemented medical report processing using DocLayNet and PaddleOCR.

•Implemented disease entity extraction using fine-tuned Bio_ClinicalBERT.

•Integrated MongoDB, Neo4j, and the RAG-based AI healthcare chatbot with safety mechanisms.

•Current implemented modules are tested and functional, while further medical intelligence modules remain for the next phase

Conclusion

•Successfully developed the core MediTra AI healthcare platform with frontend, backend, database, and workflow integration.

•Implemented automated medical report processing using DocLayNet and PaddleOCR.

•Successfully extracted disease entities using fine-tuned Bio_ClinicalBERT.

•Integrated MongoDB, Neo4j, RAG, and Gemini for patient-context-aware healthcare assistance.

- The implemented modules were successfully tested and validated, providing a strong foundation for further development.

References

[1] J. Lee, W. Yoon, S. Kim, D. Kim, S. Kim, C. H. So, and J. Kang, "BioBERT: a pre-trained biomedical language representation model for biomedical text mining," *Bioinformatics*, vol. 36, no. 4, pp. 1234–1240, 2020, doi: 10.1093/bioinformatics/btz682.

[2] B. Pfitzmann, C. Auer, M. Dolfi, A. S. Nassar, and P. W. J. Staar, "DocLayNet: A large human-annotated dataset for document-layout segmentation," in *Proc. 28th ACM SIGKDD Conf. Knowledge Discovery and Data Mining (KDD)*, 2022, pp. 3743–3751, doi: 10.1145/3534678.3539043.

[3] C.-H. Wei, Y. Peng, R. Leaman, A. P. Davis, C. J. Mattingly, J. Li, T. C. Wiegers, and Z. Lu, "Assessing the state of the art in biomedical relation extraction: Overview of the BioCreative V chemical-disease relation (CDR) task," *Database*, vol. 2016, Art. no. baw032, 2016, doi: 10.1093/database/baw032.

THANK YOU